



Programação de Aplicativos

Professor: Euclides Paim
euclidespaim@gmail.com



Módulos em Python

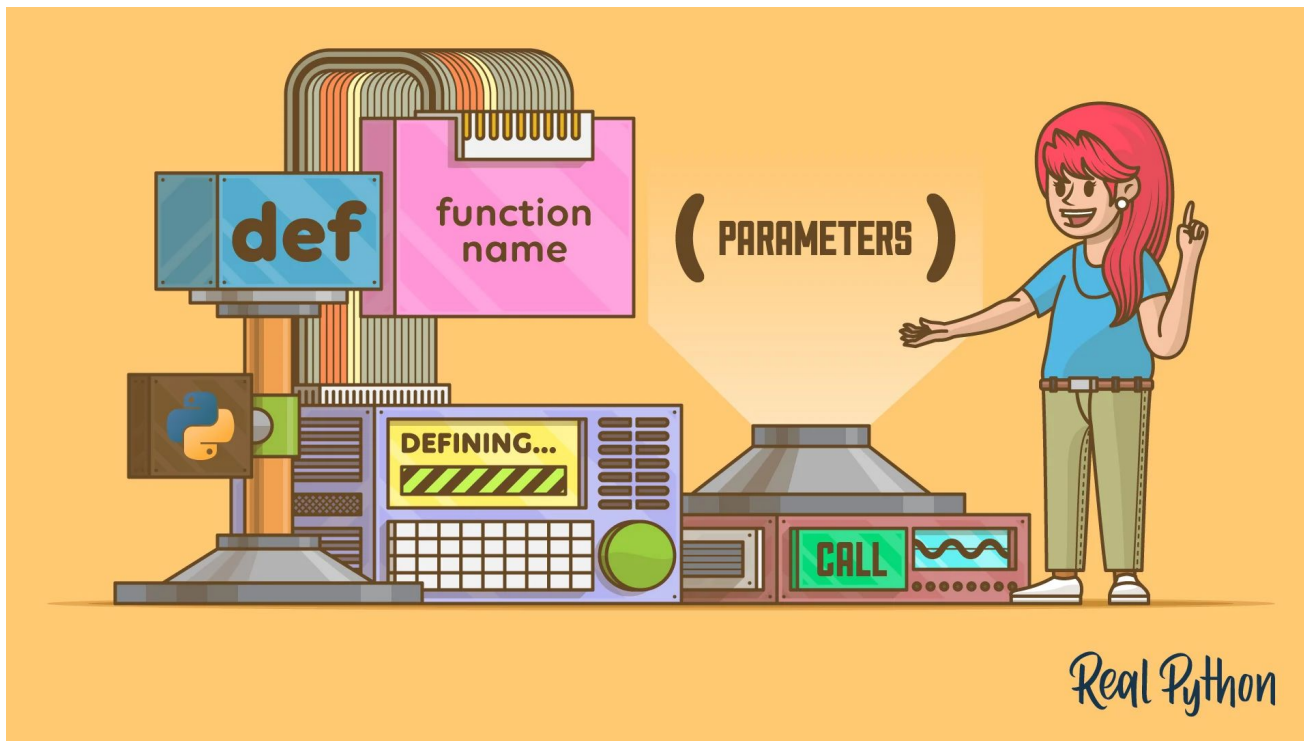
Python Modules

Professor: Euclides Paim
euclidespaim@gmail.com



Programação de Aplicativos

Módulos em Python





Programação de Aplicativos

Módulos em Python

Sumário

- Definição
- Uso
- Variáveis do módulo
- Apelidando um módulo (*Alias*)
- Módulos nativos (*built-in modules*)
- Função *dir()*
- Importando partes de um módulo (*Import from*)
- Conclusão e boas práticas





Programação de Aplicativos

Modules em Python

Modules em Python

Definição Teórica: Um módulo é o mesmo que uma biblioteca de código. É um arquivo contendo um conjunto de funções e variáveis que você deseja incluir em sua aplicação para evitar a repetição de código.

Exemplo de Sintaxe:

```
import nome_do_modulo
```

Analogia: Pense no módulo como uma caixa de ferramentas temática. Em vez de carregar todas as ferramentas do mundo soltas na mochila, você cria uma caixa específica para "Elétrica" ou "Carpintaria" e só a abre quando precisa daquelas ferramentas.



Programação de Aplicativos

Modules em Python

Criando e Usando um Módulo

- Para criar um módulo, basta salvar o código que você deseja em um arquivo com a extensão `.py`. Para utilizá-lo em outro arquivo, usamos a instrução `import` seguida pelo nome do arquivo (sem o `.py`).

```
# Arquivo de origem: ferramentas.py
def saudar(nome):
    print("Olá, " + nome)

# Arquivo principal: main.py
import ferramentas
ferramentas.saudar("Turma 202")
```

- **Analogia:** Criar um módulo é como escrever uma receita em uma folha avulsa e guardá-la em uma pasta. Usar o módulo é como abrir a pasta na página exata daquela receita quando você decide cozinhar.



Programação de Aplicativos

Modules em Python

Variáveis dentro de Módulos

- **Definição Teórica:** Os módulos não servem apenas para guardar funções. Eles também podem armazenar variáveis de qualquer tipo, como listas, tuplas, dicionários e objetos, funcionando como repositórios de dados centralizados.
- **Exemplo de Sintaxe:**

```
# Arquivo de origem: dados.py
aluno = {"nome": "Carlos", "idade": 17, "turma": "202"}

# Arquivo principal: main.py
import dados
print(dados.aluno["nome"])
```



Programação de Aplicativos

Modules em Python

Apelidando um Módulo (Alias)

- **Definição Teórica:** Você pode criar um apelido (alias) para um módulo no momento da importação utilizando a palavra-chave `as`. Isso é muito útil para encurtar nomes de arquivos longos e tornar a digitação mais rápida.
- **Exemplo de Sintaxe:**

```
import meu_modulo_de_configuracao_longo as mod  
  
# Usando o apelido curto  
mod.minha_funcao()
```




Programação de Aplicativos

Modules em Python

Módulos Nativos (*Built-in Modules*)

- **Definição Teórica:** O Python possui uma grande quantidade de módulos integrados (nativos) que vêm instalados por padrão com a linguagem. Você pode importá-los a qualquer momento sem precisar criar arquivos novos.
- **Exemplo de Sintaxe:**

```
import platform

sistema = platform.system()
print(sistema) # Exibe 'Linux', 'Windows' ou 'Darwin'
```

- **Analogia:** São como os aplicativos padrão que já vêm de fábrica quando você compra um celular novo (calculadora, relógio, bloco de notas). Você não precisa baixar nada da internet, eles já estão prontos no sistema.



Programação de Aplicativos

Modules em Python

A Função *dir()*

- **Definição Teórica:** A função embutida `dir()` é uma ferramenta de inspeção. Ela retorna uma lista contendo todos os nomes de funções, variáveis e submódulos que existem dentro de um determinado módulo (seja ele nativo ou criado por você).
- **Exemplo de Sintaxe:**

```
import platform

lista_de_recursos = dir(platform)
print(lista_de_recursos)
```



Programação de Aplicativos

Modules em Python

Importando Apenas Partes de um Módulo (*Import From*)

- **Definição Teórica:** Se você não precisa de todo o conteúdo de um módulo grande, pode escolher trazer para o seu código apenas funções ou variáveis específicas utilizando a sintaxe `from ... import ...`.
- **Exemplo de Sintaxe:**

```
# Importa APENAS a função somar, ignorando o resto do arquivo
from calculos import somar

# Quando importado assim, NÃO se usa o prefixo do módulo
resultado = somar(10, 20)
print(resultado)
```

- **Analogia:** É como ir até a biblioteca da escola e tirar xerox de apenas uma página específica de um livro de 500 páginas, em vez de pegar o livro inteiro emprestado e carregar peso desnecessário na mochila.

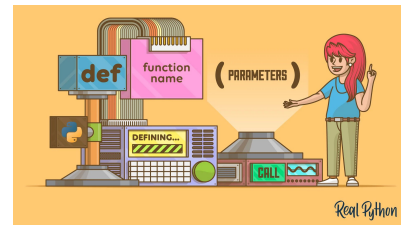


Programação de Aplicativos

Modules em Python

Conclusão e Boas Práticas

- **Regra Importante:** Quando você usa a sintaxe `from módulo import elemento`, não utilize o nome do módulo como prefixo ao chamar o elemento (chame apenas `elemento()` e não `módulo.elemento()`).
- **Organização:** Manter funções isoladas em arquivos separados facilita a manutenção do código e permite que múltiplos programadores trabalhem no mesmo projeto sem gerar conflitos.





That's all Folks!



Referências

Referências Básicas

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. ***Algoritmos: teoria e prática***. 3. ed. Rio de Janeiro: Elsevier, 2013.

SEBESTA, Robert W. ***Conceitos de linguagens de programação***. 10. ed. São Paulo: Pearson, 2018.

MANZANO, José Augusto N. G.; OLIVEIRA, Jayr Figueiredo de. ***Algoritmos: lógica para desenvolvimento de programação de computadores***. 27. ed. São Paulo: Érica, 2016.

DOWNEY, Allen B. ***Pense em Python: como pensar como um cientista da computação***. Tradução de Cássio de Souza Costa. 2. ed. São Paulo: Novatec, 2016.

Referências Complementares

IEPSEN, Edécio Fernando. ***Lógica de programação e algoritmos com JavaScript***. 2. ed. São Paulo: Novatec, 2022.

Referências na Internet

<https://www.w3schools.com/python/>